Research Software Engineering in Python for Reproducible Computational Science

Sylwester Arabas

AGH University of Krakow

2026-04-09

previously on...

- what (or who) is **RSE**?
- Python as a glue language
- unit-aware coding/plotting (**Pint**)

previously on...

- what (or who) is **RSE**?
- Python as a glue language
- unit-aware coding/plotting (**Pint**)

- **reproducibility** in computational science
- automation & reuse (DRY)

AGH

previously on...

- what (or who) is **RSE**?
- Python as a glue language
- unit-aware coding/plotting (**Pint**)

- **reproducibility** in computational science
- automation & reuse (DRY)

- Python's "duck typing" (e.g. SciPy+Pint)
- JIT-compilation in Python (e.g. Numba)

AGH

previously on...

- what (or who) is **RSE**?
- Python as a glue language
- unit-aware coding/plotting (**Pint**)

- **reproducibility** in computational science
- automation & reuse (DRY)

- Python's "duck typing" (e.g. SciPy+Pint)
- JIT-compilation in Python (e.g. Numba)

- software vs. content licensing
- open-source: copyleft/permissive
- semantic versioning
- persistent archival @Zenodo (CERN)
- Python packaging (pyproject.toml)

AGH

plan for today

- Python projects: generating documentation (from docstrings)
- Python projects: unit testing with pytest & GH Actions
- Python projects: (test.)PyPI.org uploads with GH Actions
- Definition of test-assignment and grading criteria

plan for today

- **Python projects: generating documentation (from docstrings)**
- Python projects: unit testing with pytest & GH Actions
- Python projects: (test.)PyPI.org uploads with GH Actions
- Definition of test-assignment and grading criteria

a Python function without a docstring

```
from random import random as u01
pi = lambda n: 4 * sum(abs(u01() + 1j * u01()) < 1 for _ in range(n)) / n
```

```
help(pi)
```

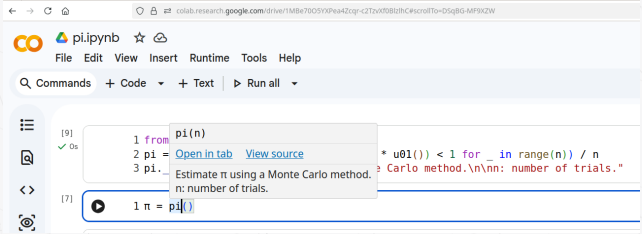
```
Help on function <lambda> in module __main__:

<lambda> lambda n
```

a Python function with a docstring

```
pi.__doc__ = "Estimate π using a Monte Carlo method.\n\nn: number of trials."
```

```
help(pi)
```

```
Help on function <lambda> in module __main__:

<lambda> lambda n
    Estimate π using a Monte Carlo method.

    n: number of trials.
```



https://github.com/hgrecco/pint/blob/master/pint/registry.py ⇝ pint.readthedocs.io

```
165      def setup_matplotlib(self, enable: bool = True) -> None:
166          """Set up handlers for matplotlib's unit support.
167
168          Parameters
169          ----------
170          enable : bool
171              whether support should be enabled or disabled (Default value = True)
172
173          """
```

https://github.com/hgrecco/pint/blob/master/pint/registry.py ⇝ pint.readthedocs.io

```
165     def setup_matplotlib(self, enable: bool = True) -> None:
166         """Set up handlers for matplotlib's unit support.
167
168         Parameters
169         ----------
170         enable : bool
171             whether support should be enabled or disabled (Default value = True)
172
173         """
```

pint.readthedocs.io/en/stable/api/base.html#pint.UnitRegistry.setup_matplotlib

setup_matplotlib(*enable: bool = True*) → None [source]

Set up handlers for matplotlib's unit support.

Parameters:

enable (bool) – whether support should be enabled or disabled (Default value = True)

automating docs deployment with pdoc, GH Actions & Pages

```yaml
name: pdoc
on:
  push:
    branches: [ main ]
jobs:
  pdoc:
    runs-on: ubuntu-latest
    permissions:
      contents: write
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install pdoc; python -m pdoc -o html mc_pi
      - uses: JamesIves/github-pages-deploy-action@v4
        with:
          folder: html
```

USER.github.io/REPO

API Documentation

pi()

built with pdoc

mc_pi

▸ View Source

def pi(n): ▾ View Source

```
3  pi = lambda n: 4 * sum(abs(u01() + 1j * u01()) < 1 for _ in range(n)) / n
```

Estimate π using a Monte Carlo method.

n: number of trials.

plan for today

- Python projects: generating documentation (from docstrings)
- **Python projects: unit testing with pytest & GH Actions**
- Python projects: (test.)PyPI.org uploads with GH Actions
- Definition of test-assignment and grading criteria

Anatomy of a test

In the simplest terms, a test is meant to look at the result of a particular behavior, and make sure that result aligns with what you would expect. Behavior is not something that can be empirically measured, which is why writing tests can be challenging.

"Behavior" is the way in which some system **acts in response** to a particular situation and/or stimuli. But exactly *how* or *why* something is done is not quite as important as *what* was done.

You can think of a test as being broken down into four steps:

1. **Arrange**
2. **Act**
3. **Assert**
4. **Cleanup**

Arrange is where we prepare everything for our test. This means pretty much everything except for the "**act**". It's lining up the dominoes so that the **act** can do its thing in one, state-changing step. This can mean preparing objects, starting/killing services, entering records into a database, or even things like defining a URL to query, generating some credentials for a user that doesn't exist yet, or just waiting for some process to finish.

Act is the singular, state-changing action that kicks off the **behavior** we want to test. This behavior is what carries out the changing of the state of the system under test (SUT), and it's the resulting changed state that we can look at to make a judgement about the behavior. This typically takes the form of a function/method call.

Assert is where we look at that resulting state and check if it looks how we'd expect after the dust has settled. It's where we gather evidence to say the behavior does or does not align with what we expect. The `assert` in our test is where we take that measurement/observation and apply our judgement to it. If something should be green, we'd say `assert thing == "green"`.

a hello-world unit test

```
%%file test_pi.py

import random
import math
import mc_pi

def test_pi():
    # arrange
    random.seed(44)

    # act
    value = mc_pi.pi(n=10000)

    # assert
    assert abs(math.pi - value) / math.pi < .001
```

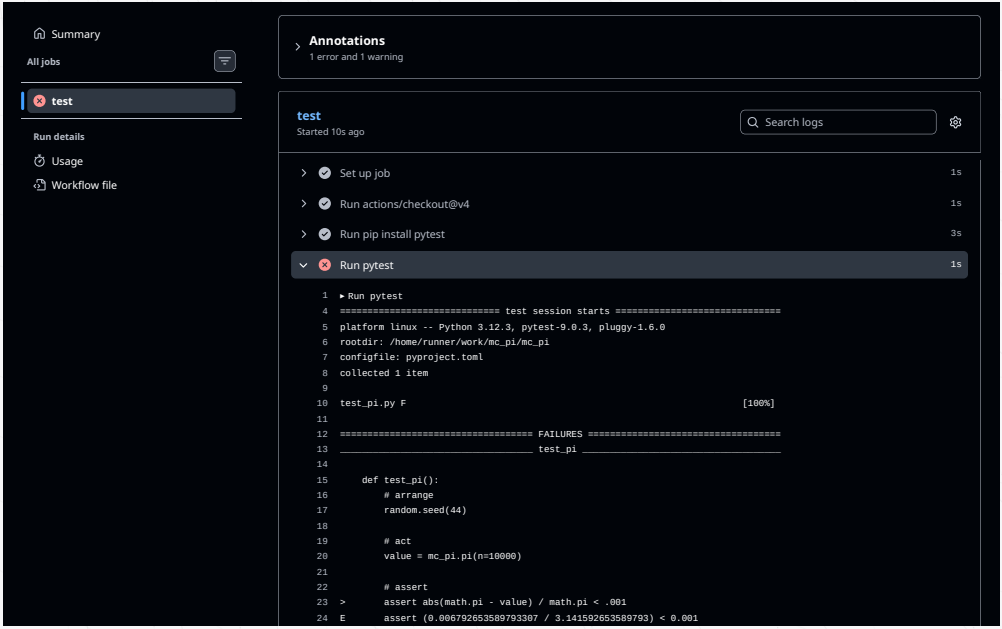
invoking pytest

```
!pytest --quiet test_pi.py
```

```
F
================================================================================
_______________________________________________________________________________

    def test_pi():
        # arrange
        random.seed(44)

        # act
        value = mc_pi.pi(n=10000)

        # assert
>       assert abs(math.pi - value) / math.pi < .001
E       assert (0.006792653589793307 / 3.141592653589793) < 0.001
```

using GH Actions for a Continuous-Integration (CI) setup

```yaml
name: pytest

on: [push]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install pytest
      - run: pytest
```


Summary
All jobs
test
Run details
Usage
Workflow file
Annotations
1 error and 1 warning
test
Started 10s ago
Search logs
Set up job 1s
Run actions/checkout@v4 1s
Run pip install pytest 3s
Run pytest 1s
▸ Run pytest
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /home/runner/work/mc_pi/mc_pi
configfile: pyproject.toml
collected 1 item
test_pi.py F [100%]
=================================== FAILURES ===================================
_____ test_pi _____
def test_pi():
# arrange
random.seed(44)
# act
value = mc_pi.pi(n=10000)
# assert
> assert abs(math.pi - value) / math.pi < .001
E assert (0.0067926535897933 07 / 3.141592653589793) < 0.001

„tests turn assumptions into guarantees’ ’

„tests turn assumptions into guarantees’ ’

- executable specification

„tests turn assumptions into guarantees’ ’

- executable specification
- regression safety net

AGH

„tests turn assumptions into guarantees’ ’

- executable specification
- regression safety net
- automated validation

AGH

„tests turn assumptions into guarantees’ ’

- executable specification
- regression safety net
- automated validation
- progress indicator

AGH

„tests turn assumptions into guarantees’ ’

- executable specification
- regression safety net
- automated validation
- progress indicator
- documentation & onboarding aid

AGH

„tests turn assumptions into guarantees’ ’

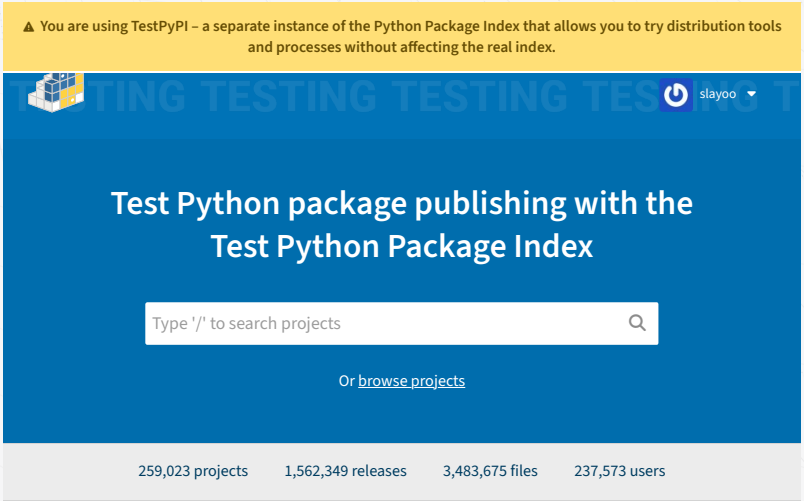
- executable specification
- regression safety net
- automated validation
- progress indicator
- documentation & onboarding aid
- design feedback (modularity)

AGH

plan for today

- Python projects: generating documentation (from docstrings)
- Python projects: unit testing with pytest & GH Actions
- **Python projects: (test.)PyPI.org uploads with GH Actions**
- Definition of test-assignment and grading criteria

test.pypi.org



```yaml
on:
  release:
    types: [published]
jobs:
  publish:
    permissions:
      id-token: write
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install build; python -m build
      - uses: pypa/gh-action-pypi-publish@release/v1.13
        with:
          repository_url: https://test.pypi.org/legacy/
          attestations: false
```

plan for today

- Python projects: generating documentation (from docstrings)
- Python projects: unit testing with pytest & GH Actions
- Python projects: (test.)PyPI.org uploads with GH Actions
- **Definition of test-assignment and grading criteria**

AGH

Test assignment (lecture material)

Create a (hello-world) public Python project on GitHub with:

Test assignment (lecture material)

Create a (hello-world) public Python project on GitHub with:

- usage of either `Numba` or `Pint` (or both)

Test assignment (lecture material)

Create a (hello-world) public Python project on GitHub with:

- usage of either `Numba` or `Pint` (or both)
- CI-automated unit test[s] (e.g., using pytest)

AGH

Test assignment (lecture material)

Create a (hello-world) public Python project on GitHub with:

- usage of either `Numba` or `Pint` (or both)
- CI-automated unit test[s] (e.g., using pytest)
- `pyproject.toml`-based package metadata (incl. dependencies, license, version)

Test assignment (lecture material)

Create a (hello-world) public Python project on GitHub with:

- usage of either `Numba` or `Pint` (or both)
- CI-automated unit test[s] (e.g., using pytest)
- `pyproject.toml`-based package metadata (incl. dependencies, license, version)
- automated docs deployment (e.g., to GitHub Pages)

AGH

Test assignment (lecture material)

Create a (hello-world) public Python project on GitHub with:

- usage of either `Numba` or `Pint` (or both)
- CI-automated unit test[s] (e.g., using pytest)
- `pyproject.toml`-based package metadata (incl. dependencies, license, version)
- automated docs deployment (e.g., to GitHub Pages)
- automated on-release uploads to `test.pypi.org`

AGH

Test assignment (lecture material)

Create a (hello-world) public Python project on GitHub with:

- usage of either `Numba` or `Pint` (or both)
- CI-automated unit test[s] (e.g., using pytest)
- `pyproject.toml`-based package metadata (incl. dependencies, license, version)
- automated docs deployment (e.g., to GitHub Pages)
- automated on-release uploads to `test.pypi.org`
- `colab.google`/`mybinder.org`-ready Jupyter notebook with a demo (incl. installation)

AGH

Test assignment (lecture material)

Create a (hello-world) public Python project on GitHub with:

- usage of either `Numba` or `Pint` (or both)
- CI-automated unit test[s] (e.g., using pytest)
- `pyproject.toml`-based package metadata (incl. dependencies, license, version)
- automated docs deployment (e.g., to GitHub Pages)
- automated on-release uploads to `test.pypi.org`
- `colab.google`/`mybinder.org`-ready Jupyter notebook with a demo (incl. installation)

Some extra suggestions:

AGH

Test assignment (lecture material)

Create a (hello-world) public Python project on GitHub with:

- usage of either `Numba` or `Pint` (or both)
- CI-automated unit test[s] (e.g., using pytest)
- `pyproject.toml`-based package metadata (incl. dependencies, license, version)
- automated docs deployment (e.g., to GitHub Pages)
- automated on-release uploads to `test.pypi.org`
- `colab.google`/`mybinder.org`-ready Jupyter notebook with a demo (incl. installation)

Some extra suggestions:

- use `setuptools_scm` for automated git-coupled versioning

AGH

Test assignment (lecture material)

Create a (hello-world) public Python project on GitHub with:

- usage of either `Numba` or `Pint` (or both)
- CI-automated unit test[s] (e.g., using pytest)
- `pyproject.toml`-based package metadata (incl. dependencies, license, version)
- automated docs deployment (e.g., to GitHub Pages)
- automated on-release uploads to `test.pypi.org`
- `colab.google`/`mybinder.org`-ready Jupyter notebook with a demo (incl. installation)

Some extra suggestions:

- use `setuptools_scm` for automated git-coupled versioning
- setup automated code-coverage using (e.g., using `codecov`)

Test assignment (lecture material)

Create a (hello-world) public Python project on GitHub with:

- usage of either `Numba` or `Pint` (or both)
- CI-automated unit test[s] (e.g., using pytest)
- `pyproject.toml`-based package metadata (incl. dependencies, license, version)
- automated docs deployment (e.g., to GitHub Pages)
- automated on-release uploads to `test.pypi.org`
- `colab.google`/`mybinder.org`-ready Jupyter notebook with a demo (incl. installation)

Some extra suggestions:

- use `setuptools_scm` for automated git-coupled versioning
- setup automated code-coverage using (e.g., using `codecov`)

send a colab.google or mybinder.org link via e-mail by May 10

AGH

take-home messages:

take-home messages:

- **not automated** $\rightsquigarrow$ not reproducible $\rightsquigarrow$ **not science**

take-home messages:

- **not automated** $\rightsquigarrow$ not reproducible $\rightsquigarrow$ **not science**
- **not automated** $\rightsquigarrow$ not scalable $\rightsquigarrow$ **not productizable**

take-home messages:

- **not automated** $\rightsquigarrow$ not reproducible $\rightsquigarrow$ **not science**
- **not automated** $\rightsquigarrow$ not scalable $\rightsquigarrow$ **not productizable**

- not unit-tested
  $\rightsquigarrow$ no robust way to check if it still works
  $\rightsquigarrow$ fear-driven development, unrefactorable code
  $\rightsquigarrow$ **technical debt**

AGH

take-home messages:

- **not automated** $\rightsquigarrow$ not reproducible $\rightsquigarrow$ **not science**
- **not automated** $\rightsquigarrow$ not scalable $\rightsquigarrow$ **not productizable**

- not unit-tested
  - $\rightsquigarrow$ no robust way to check if it still works
  - $\rightsquigarrow$ fear-driven development, unrefactorable code
  - $\rightsquigarrow$ **technical debt**
- versioned & packaged
  - $\rightsquigarrow$ unambiguous & unassisted access/deployment

AGH

Thank you for your attention!

slides and other course resources at
https://github.com/open-atmos-krk/rse-python-course